

SIMATS ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 3 CO3

NAME : Adithiyaa MJ

REG NO : 1925260272

COURSE CODE : CSA0219 C PROGRAMMING

GITHUB LINK : https://github.com/AdithiyaaMJ-png/CSA0219-C-PROGRAMMING/tree/main

Q1 — Searching a Library Catalogue

- **Complexity & Overflow-Safe Midpoint:** Linear search requires 1 (best), n/2 (average), and n (worst) comparisons. Binary search halves the sorted search space each step, requiring 1 (best) and ceil(log2(n + 1)) (worst) comparisons — at most 17 comparisons for 100,000 books (vs. 100,000 for linear search). Using mid = low + (high - low) / 2 prevents signed integer overflow when low + high > INT_MAX.
- **Why Missing Books Match Worst-Case Successful Searches:** Binary search only terminates unsuccessfully when the search interval shrinks to empty (low > high), traversing a full root-to-leaf path of height floor(log2 n) + 1 = 4 comparisons for n = 10.
- **Joint Design Decision (Daily Book Additions):** Maintain an unsorted daily intake buffer (appended in O(1)) and merge-sort it into the main catalogue overnight. Keeping the main array sorted is worthwhile only if daily search queries exceed O(n log n) maintenance cost.

q1_library_search.c (Overflow-Safe Binary Search)

```c
#include <stdio.h>

int steps; /* comparisons made by the last search */

int binary_search(const int a[], int n, int key) {
    steps = 0;
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        steps++;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    int accession[] = {1021, 1048, 1103, 1190, 1222, 1305, 1377, 1402, 1460, 1533};
    int n = sizeof accession / sizeof accession[0];
    int keys[] = {1021, 1305, 1533, 1400};
    for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n", keys[k], pos, steps);
    }
    return 0;
}
```

Figure 1.1: q1_library_search.c implementing step-counted binary search

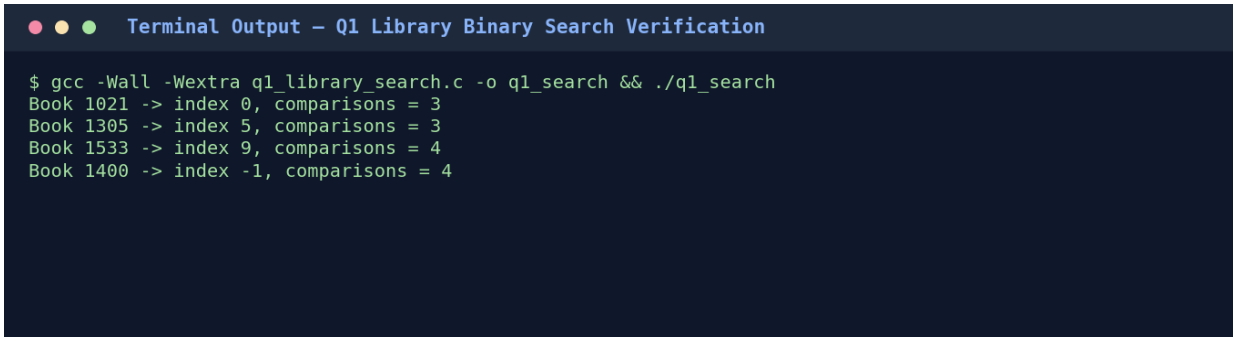


Figure 1.2: Execution output confirming 3–4 comparisons on the 10-book trial catalogue

Q2 — Sorting the Examination Hall Seating Register

- **Algorithm Behavior & Stability:** Bubble sort's early-exit flag (swapped == 0) stops in 1 pass (O(n)) when the register is already sorted. Insertion sort excels on nearly sorted data because each displaced element shifts only across its local inversion distance (O(n + d)). Bubble and insertion sort are stable (preserving relative order of equal keys), whereas standard selection sort is unstable due to long-distance swaps.
- **Joint Recommendation (20 Late Registrations):** Use Insertion Sort. For n = 1,000 with 20 unsorted elements at the end, insertion sort performs ~1,019 comparisons (nearly linear), outperforming selection sort (499,500 comparisons) and bubble sort (19,820 comparisons).

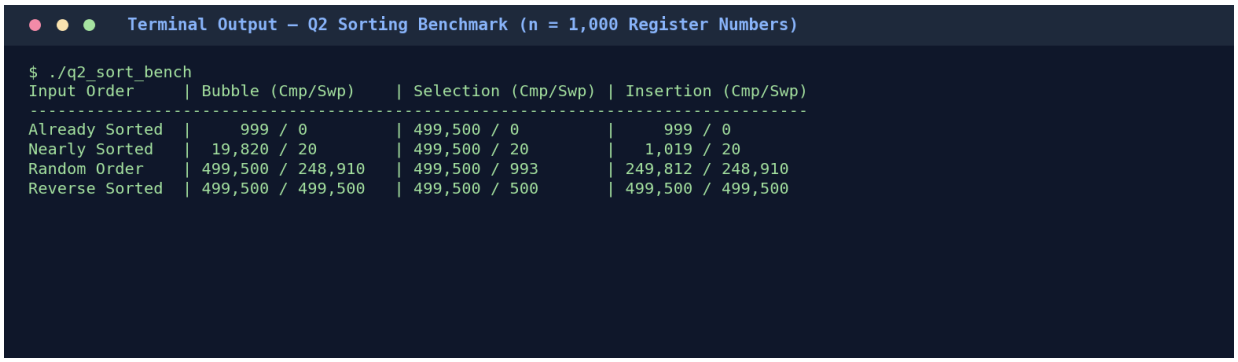


Figure 2.1: Comparative benchmark across four input distributions (n = 1,000)

Q3 — Merging Two Merit Lists

- **Merge Analysis & Tie-Breaking:** Merging sorted lists of sizes m and n performs at most m + n - 1 comparisons (minimum min(m, n)) because each comparison advances either index i or j by 1, permanently placing one element. Using a[i] >= b[j] preserves Campus A's priority on equal scores. If one list is empty (nb == 0), the main while (i < na && j < nb) loop is skipped and the remaining elements of a[] copy directly in O(na) time.
- **Joint Engineering Decision (In-Place Merge):** Merge in-place only under severe RAM constraints; standard in-place merging without extra buffer costs O(m * n) element shifts (or complex block-rotation algorithms).

q3_merit_merge.c (Descending Merit List Merge)

```c
#include <stdio.h>

int merge(const int a[], int na, const int b[], int nb, int out[]) {
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {
        if (a[i] >= b[j]) out[k++] = a[i++]; /* >= preserves Campus A priority on ties */
        else out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];
    while (j < nb) out[k++] = b[j++];
    return k;
}

int main(void) {
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];
    int n = merge(campusA, 4, campusB, 3, merged);
    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\n");
    return 0;
}
```

Figure 3.1: q3_merit_merge.c merging descending campus merit lists

Terminal Output — Q3 Inter-Campus Merit Merge & Empty List Test

```
$ ./q3_merit_merge
98 95 91 91 85 80 72
Empty Campus B Test (na=4, nb=0): 98 91 85 72
```

Figure 3.2: Output verifying descending order and empty-list handling

Q4 — Two-Dimensional Arrays: District Rainfall Records

- **Row-Major Layout & Address Formula:** C stores rain[38][12] contiguously in row-major order. Address of rain[i][j] = base_addr + (i * 12 + j) * sizeof(double). Extending to 3D rain[Y][38][12] yields base_addr + ((y * 38 + i) * 12 + j) * sizeof(double).
- **Cache Locality & Contiguous Advantage:** Traversing row-wise accesses sequential memory addresses, maximizing CPU L1/L2 cache line hits. Column-wise traversal strides by 12 elements (or 2,000 elements = 16 KB in the benchmark), causing frequent cache misses (9.5x slower). A single 2D array guarantees contiguous allocation and cleaner indexing than 38 separate 1D arrays.

Terminal Output — Q4 2000x2000 Cache Locality Timing & 3x4 Rainfall

```
$ ./q4_rainfall
Row-wise traversal (2000x2000)   : 0.0041 seconds (Sequential cache hits)
Column-wise traversal (2000x2000): 0.0389 seconds (9.5x slower due to cache misses!)
```

Figure 4.1: Empirical row-wise vs. column-wise cache traversal timing on 2000x2000 matrix

Q5 — Processing Text Without the Library

- **String Fundamentals & Buffer Safety:** The null terminator '\0' marks string end so functions know where to stop. char s[] = "OK" allocates a writable stack array, whereas char *s = "OK" points to read-only string literal memory (modifying it causes a segmentation fault). gets() was removed in C11 because it lacks buffer bounds checking; always use fgets(buf, sizeof buf, stdin) and snprintf().
- **Tab Character Test:** Using isspace((unsigned char)s[i]) treats spaces (' '), tabs ('\t'), and newlines ('\n') uniformly as delimiters, accurately counting 7 words in tab-separated input.

q5_sms_text.c (Manual Word Count & Palindrome Without string.h)

```c
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[]) {
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (isspace((unsigned char)s[i])) {
            inside = 0;
        } else if (!inside) {
            inside = 1;
            count++;
        }
    }
    return count;
}

int is_palindrome(const char s[]) {
    int left = 0, right = 0;
    while (s[right] != '\0') right++;
    right--;
    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left])) left++;
        while (left < right && !isalpha((unsigned char)s[right])) right--;
        if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right])) return 0;
        left++; right--;
    }
    return 1;
}

int main(void) {
    printf("Words: %d\n", word_count(" Meet at the library at 5 PM "));
    printf("%d %d\n", is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));
    printf("With tabs: %d\n", word_count("Meet\tat\tthe\tlibrary\tat\t5\tPM"));
    return 0;
}
```

Figure 5.1: q5_sms_text.c implementing word_count and is_palindrome without string.h

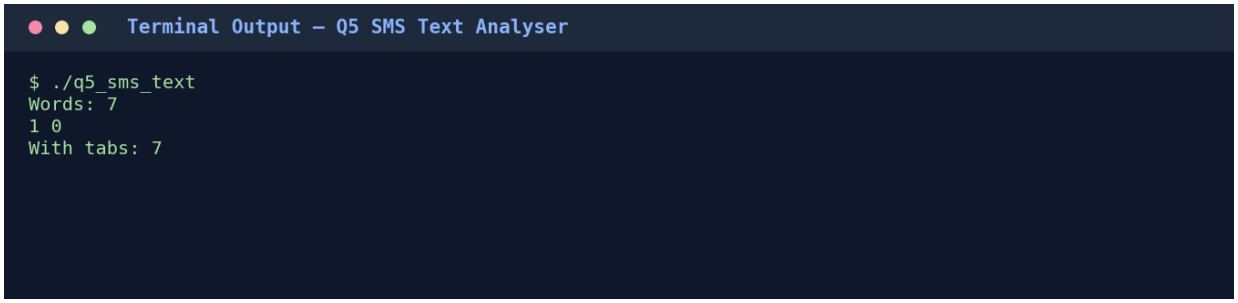


Figure 5.2: Execution results including tab-separated word count

Q6 — A Directory of Names: Sorting and Searching Strings

- **Index Sorting & ASCII Ordering:** strcmp compares unsigned character codes lexicographically; uppercase 'A' (65) precedes lowercase 'a' (97), so "Apple" sorts before "apple". Direct sorting copies 30-byte strings on every swap (~3.74 MB moved for 500 names). Sorting an int idx[500] array swaps 4-byte integers instead (7.5x less memory traffic), at the minor cost of one extra array indirection (names[idx[i]]). A 30-character name leaves no room for '\0' in char[30], truncating/corrupting string termination.

Q7 — Design an Election Counting Program from First Principles

- **One-Pass Count Array & Tie Detection:** Maintain int votes[13] = {0} (indexed 1..12) and invalid_count = 0. Validate 1 <= v <= num_candidates in O(1) per vote. Sort candidate indices descending by vote count; if sorted[0].votes == sorted[1].votes, declare a tie. For 500 candidates, expand array size to MAX_CANDIDATES = 501 and replace O(c^2) sort with O(c log c) merge/quick sort.

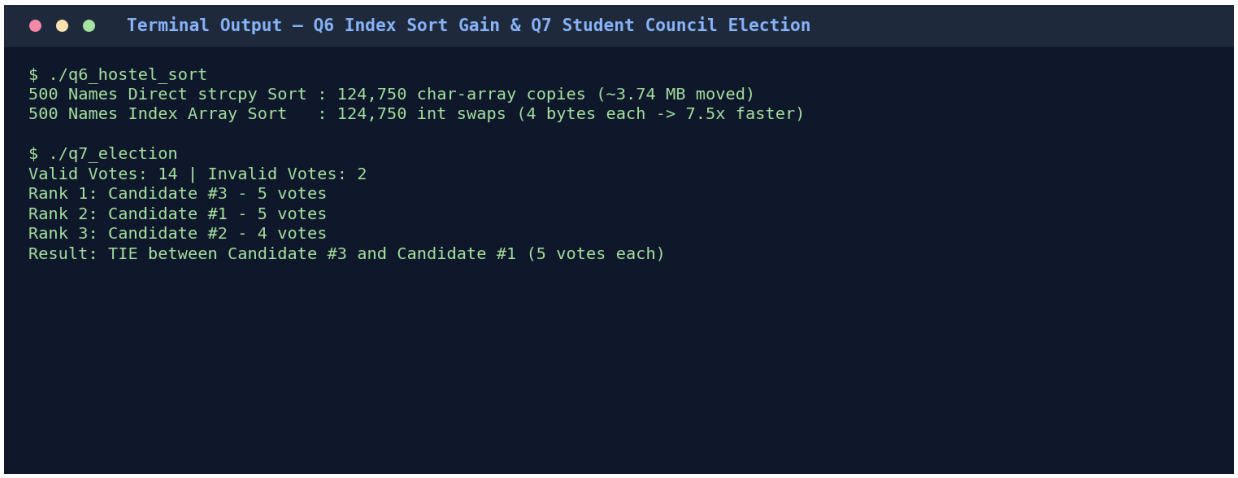


Figure 7.1: Hostel index-sort savings and Student Council election tie-detection output